04 分治法 • 獨立詳解

04-E01 最大子陣列的狀態與區間

推導

每個以i結尾的非空區間不是單元素Ai，就是某個以i−1結尾的區間接Ai。後者最佳必取先前最佳，故end(i)=max(Ai,end(i−1)+Ai)，best(i)=max(best(i−1),end(i))，初值皆A1。

i	1	2	3	4	5	6	7
A[i]	5	−9	2	6	−1	4	−3
end	5	−4	2	8	7	11	8
best	5	5	5	8	8	11	11

第3格重新開始，之後延伸到第6格得2+6−1+4=11。全負時原規格必選其中最大單元素；允許空區間才可把全域答案與0比較、回空區間。

結論與檢核

最佳區間3..6，和11。每輪常數次工作，Θ(n)時間、O(1)額外空間；非空規格不能把初值best設0。

04-E02 遞迴樹與同一常數歸納

推導

令n=3^L。每個內層有3^j個大小n/3^j的節點，層成本n；L個內層共nL。葉3^L=n，每個成本1，故T(n)=n(L+1)。

取上界U(n)=n(log₃n+1)。基例U1=1=T1；代入3U(n/3)+n=n(log₃n−1+1)+n=U(n)，實際等號成立。若猜T≤cn，則代入得到cn+n，不能推出原先同一個cn；把c換為c+1會使歸納假設不閉合。

結論與檢核

精確解n(1+log₃n)，緊界Θ(n lg n)。n=9的層成本9、9、9，總27。

04-E03 對數平方的分水邊界

推導

a=9,b=3,p=2，f/n^p=(lg n)²無界但比任何固定n^ε小，故非經典情形2或3，也非情形1。n=3^L時第j層為n²(lg3)²(L−j)²；令r=L−j，內層和為n²(lg3)²Σr²。

$\sum_{r=1}^{L} r^2 = L(L+1)(2L+1)/6 = \Theta(L^3)$。葉成本Θ(n²)低一個對數三次方；取整在常數底層以上保持同階。

04 分治法 • 獨立詳解 （續）

結論與檢核

Θ(n²lg³n)。這是由層和證明的對數擴充，不是把「f較大」當成情形3。

04-E04 不等分割與擾動

推導

採正文引用的連續定理：固定正ai、0<bi<1，正且有界基例，非負可微g且導數至多多項式增長，擾動hi=O(x/log²x)，並使child在[0,ρx]、ρ<1。此例a1=a2=1、b1=1/3、b2=2/3、h=0、g=x，全部符合；基例閾值取足夠大。

F(p)=(1/3)^p+(2/3)^p連續且嚴格遞減，p→−∞時趨∞、p→∞時趨0，因此F=1有唯一實根。p=1給1，所以唯一根1。代入得

$$T(x) = \Theta\left(x\left[1 + \int_1^x \frac{u}{u^2}\,du\right]\right) = \Theta(x(1+\ln x)).$$

第二例第一child實為3n/4，n/4不是O(n/log²n)擾動，必改比例3/4、1/2。新的唯一根由(3/4)^p+(1/2)^p=1求得p≈1.507126591639。積分是∫ 1^x u^(−p)du=(1−x^(1−p))/(p−1)=Θ(1)，故T=Θ(x^p)。

結論與檢核

第一例Θ(n lg n)，第二例Θ(n^1.507126591639…)；後者的1.507…只是數值近似，嚴格指數由唯一根定義。子問題比例與小擾動不能混用。

04-E05 七乘積的回收不能交換順序

推導

依序M1=(2+4)(5+1)=36，M2=(3+4)5=35，M3=2(2−1)=2，M4=4(−2−5)=−28，M5=(2−1)1=1，M6=(3−2)(5+2)=7，M7=(−1−4)(−2+1)=5。

C11=36−28−1+5=12，C12=2+1=3，C21=35−28=7，C22=36−35+2+7=10。一般區塊上C22=(ae+ah+de+dh)−(ce+de)+(af−ah)+(ce+cf−ae−af)=cf+dh。每個相消的項都以相同左右因子順序出現。

結論與檢核

C=[[12,3],[7,10]]，直接三層乘法核對相同。純量例雖可交換乘法，但歸納中a至h是矩陣，de不能一般改成ed；否則相消證明失效。

04 分治法 • 獨立詳解 （續）

04-E06 不掃描的最大子陣列合併

推導

設相鄰L、R的摘要為(t,p,s,b)，分別總和、最佳非空prefix、suffix、子陣列和。合併為

$$t = t_L + t_R, \quad p = \max(p_L, t_L + p_R), \quad s = \max(s_R, s_L + t_R), \quad b = \max(b_L, b_R, s_L + p_R).$$

prefix要麼全在L，要麼包含整個L再接R的prefix；suffix反向同理；best以左、右、跨線三類覆蓋。基例四值皆Ai，所有最優端點可隨選項傳遞。歸納即證全部摘要正確，不允許空區間時全負也成立。

結論與檢核

每個內節點O(1)合併，T(n)=T(⌊n/2⌋)+T(⌈n/2⌉)+Θ(1)=Θ(n)，有n葉及n−1內節點。CROSS版本每層重新掃描，回傳資訊較少，故其Θ(n lg n)分析仍正確。

04-E07 強化假設支付低階成本

推導

直接cn^p因5(n/2)^p=n^p，代入得cn^p+n²，無法≤cn^p。改猜cn^p−dn²，代入得到cn^p−(5d/4)n²+n²；要≤cn^p−dn²需d≥4。取d=4、c=5，基例c−d=1=T1，代入也恰取等號。

由展開或歸納得精確式T(n)=5n^p−4n²。又每個葉成本1，共n^p個葉，給T≥n^p；上述式給T≤5n^p，所以雙向界成立。

結論與檢核

Θ(n^(lg5))。n=2時5・5−4・4=9，與5T1+4=9一致。較強假設產生每個child的低階扣項，才能支付父節點成本。

04-E08 取整與基例的生命線

推導

n=2時⌈10/6⌉=2，會自我呼叫；n=3、4、5也有相同問題。因此只給T1不足。可令1≤n≤6皆為固定正常數；n≥7時⌈5n/6⌉≤n−1，會終止。

以b=6/5寫q(j+1)=⌈qj/b⌉，若|qj−n/b^j|≤C，下一層誤差≤C/b+1；取C=6即可閉合。為直接以深度界避開誤差正好碰基例的情況，也可將基例加大到n≤12，不改漸近階。於n/b^j落入固定區間時，即在lg_b n+O(1)層內終止；反向qj≥n/b^j給Ω(lg n)層。

04 分治法 • 獨立詳解 （續）

結論與檢核

修正後Θ(lg n)。基例6或12都可；後者讓加法誤差界的上界證明更直接，不能把只給T1的未定義遞迴當成已解。

04-E09 正則性不能省略

推導

每個n都有f≥n^2，對p=1是ε=1的多項式分離。但偶數L時，n/2的指數為奇數，因此T(n)≥2f(n/2)=n^3/4；當前f(n)=n^2，比例至少n/4趨∞。若存在T≤Cf，便須所有這些n都滿足n/4≤C，矛盾。

同時2f(n/2)/f(n)=n/4，不存在固定c<1使正則性在所有充分大輸入成立。保留正基例與非負成本也無法修復缺失的正則性。

結論與檢核

T非Θ(f)。這是缺條件的反例，並未違反主定理。

04-E10 比多項式分離更細的邊界

推導

令n=2^L，停止於固定L0≥1。p=1。對j層，分別為n/(L−j)與n/$(L-j)^2$，令r=L−j，內層和為nΣr^(−1)與nΣr^(−2)，r從固定正整數到L。

調和和Θ(lg L)，平方倒數和Θ(1)；兩者都另有Θ(n)葉成本。故第一個n lg L主導，第二個與葉同階。因L=lg n，必把第二次log還原。

結論與檢核

依序Θ(n lg lg n)、Θ(n)。兩者均非經典情形1，因任何固定ε>0都不能使log分母成為n^ε的分離。

04-E11 參數複製改變演算法成本

推導

view版每次只比較中點及傳索引：V(n)=V(⌊n/2⌋)+Θ(1)，最壞Θ(lg N)。只複製當前區間：C(n)=C(⌊n/2⌋)+Θ(n)，沿單一路徑成本N+N/2+⋯=Θ(N)。每次複製原始N：F(n;N)=F(⌊n/2⌋;N)+Θ(N)，最壞有Θ(lg N)層，所以Θ(N lg N)。

第三例的N在遞迴中保持固定，不隨n縮小；若把Θ(N)寫成Θ(n)，就錯換了成本模型。遞迴失敗搜尋或最深葉可達相應最壞深度，下界也成立。

04 分治法 • 獨立詳解 （續）

結論與檢核

依序Θ(lg N)、Θ(N)、Θ(N lg N)。這些界是最壞情形；根即命中時三種成本可分別為常數、Θ(N)、Θ(N)。

04-E12 換變數後必須還原

推導

令m=lg n=2^k，S(m)=T(2^m)，則S1=T2=1，S(m)=3S(m/2)+m。分水指數p=lg3>1，m比m^p多項式小，所以S(m)=Θ(m^p)。也可展開：第j層工作m(3/2)^j，葉3^k=m^p，幾何和同階。

結論與檢核

T(n)=Θ((lg n)^(lg3))，域為題目指定的n=2^(2^k)。Θ(n^(lg3))把新變數m誤當原變數n；不能在還原前就改回同名符號。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

04 分治法 · 獨立詳解 (續)

原創驗算程式附錄: akra_bazzi.py

```
"""Check the arithmetic in the Akra-Bazzi supplement."""

from math import log


TERMS = ((1.0, 1.0 / 3.0), (1.0, 2.0 / 3.0))


def characteristic(p, terms=TERMS):
    return sum(coefficient * fraction**p for coefficient, fraction in terms)


def solve_exponent(terms=TERMS):
    lo, hi = -20.0, 20.0
    for _ in range(200):
        mid = (lo + hi) / 2.0
        if characteristic(mid, terms) > 1.0:
            lo = mid
        else:
            hi = mid
    return (lo + hi) / 2.0


def integral_for_g_power(n, p, exponent):
    if abs(exponent - p) < 1e-12:
        return n**p * log(n)
    return (n**exponent - n**p) / (exponent - p)

def master_case_one_bound(a, b, epsilon, n_power):
    p = log(a) / log(b)
    ratio = a / b ** (p - epsilon)
    depth = int(log(n_power, b))
    level_sum = sum(
        n_power ** (p - epsilon) * ratio**level
        for level in range(depth + 1)
    )
    return p, ratio, depth, level_sum
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

04 分治法 · 獨立詳解 （續）

```
def main():
    balanced_unequal = solve_exponent()
    assert abs(balanced_unequal - 1.0) < 1e-12
    assert abs(integral_for_g_power(64, 1.0, 1.0) - 64 * log(64)) < 1e-10

    # Keeping b=1/2 for the first call is invalid after a perturb of n/4.
    naive = ((1.0, 0.5), (1.0, 0.5))
    corrected = ((1.0, 0.75), (1.0, 0.5))
    naive_p = solve_exponent(naive)
    corrected_p = solve_exponent(corrected)
    assert abs(naive_p - 1.0) < 1e-12
    assert corrected_p > 1.4
    case_one_p, case_one_ratio, case_one_depth, case_one_sum = master_case_one_bound(
        2.0, 2.0, 0.5, 1024
    )
    assert case_one_p == 1.0
    assert case_one_ratio > 1.0
    assert abs(case_one_ratio**case_one_depth - 1024**0.5) < 1e-10
    assert case_one_sum <= 4 * 1024
    zero_t, zero_g, invalid_positive_lower = 0, 0, 1024
    assert zero_t == zero_g == 0 and invalid_positive_lower > zero_t
    print(
        "PASS "
        f"unequal_p={balanced_unequal:.12f} "
        f"integral_64={integral_for_g_power(64, 1.0, 1.0):.6f} "
        f"invalid_h_naive_p={naive_p:.12f} "
        f"corrected_base_p={corrected_p:.12f} "
        f"master_case1_ratio={case_one_ratio:.12f} "
        f"master_case1_sum={case_one_sum:.6f} "
        "zero_counterexample=T=0,g=0"
    )


if __name__ == "__main__":
    main()
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

04 分治法 · 獨立詳解 （續）

原創驗算程式附錄: divide_conquer_checks.py

```
"""Independent finite checks of Chapter 4 calculations; not proofs of asymptotic bounds."""
from itertools import product
from math import log,ceil
from random import Random
from collections import Counter

SAMPLE=[-2,1,-3,4,-1,2,1,-5,4]

def crossing(a,lo,mid,hi):
    total=0;left=float('-inf');li=mid
    for i in range(mid,lo-1,-1):
        total+=a[i]
        if total>left:left=total;li=i
    total=0;right=float('-inf');rj=mid+1
    for j in range(mid+1,hi):
        total+=a[j]
        if total>right:right=total;rj=j
    return li,rj+1,left+right

def divide(a,lo=0,hi=None):
    hi=len(a) if hi is None else hi
    if hi-lo==1:return lo,hi,a[lo]
    mid=(lo+hi-1)//2
    return max((divide(a,lo,mid+1),divide(a,mid+1,hi),crossing(a,lo,mid,hi)),key=lambda
t:t[2])

def summary(a):
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

04 分治法 • 獨立詳解 （續）

```
    if len(a)==1:return (a[0],)*4
    mid=len(a)//2;t1,p1,s1,b1=summary(a[:mid]);t2,p2,s2,b2=summary(a[mid:])
    return t1+t2,max(p1,t1+p2),max(s2,s1+t2),max(b1,b2,s1+p2)

def naive(a,b):return [[sum(x*y for x,y in zip(row,col)) for col in zip(*b)] for row in a]
def add(a,b,sign=1):return [[x+sign*y for x,y in zip(r,s)] for r,s in zip(a,b)]
def split(a):
    m=len(a)//2
    return [r[:m] for r in a[:m]],[r[m:] for r in a[:m]],[r[:m] for r in a[m:]],[r[m:] for r
in a[m:]]
def strassen(A,B):
    if len(A)==1:return [[A[0][0]*B[0][0]]]
    a,b,c,d=split(A);e,f,g,h=split(B)
    M1=strassen(add(a,d),add(e,h));M2=strassen(add(c,d),e)
    M3=strassen(a,add(f,h,-1));M4=strassen(d,add(g,e,-1))

M5=strassen(add(a,b),h);M6=strassen(add(c,a,-1),add(e,f));M7=strassen(add(b,d,-1),add(g,h))
    c11=add(add(add(M1,M4),M5,-1),M7);c12=add(M3,M5)
    c21=add(M2,M4);c22=add(add(add(M1,M2,-1),M3),M6)
    return [r+s for r,s in zip(c11,c12)]+[r+s for r,s in zip(c21,c22)]

def check():
    arrays=0
    for n in range(1,8):
        for t in product(range(-2,3),repeat=n):
            brute=max(sum(t[i:j]) for i in range(n) for j in range(i+1,n+1))
            lo,hi,v=divide(t)
            assert v==brute==sum(t[lo:hi])==summary(t)[3];arrays+=1
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

04 分治法 · 獨立詳解 （續）

```
    assert crossing(SAMPLE,0,4,9)==(3,7,6)
    rng=Random(427);matrices=0
    for n in [1,2,4,8]:
        for _ in range(50):
            A=[[rng.randrange(-6,7) for _ in range(n)] for _ in range(n)]
            B=[[rng.randrange(-6,7) for _ in range(n)] for _ in range(n)]
            assert strassen(A,B)==naive(A,B);matrices+=1
    assert strassen([[2,-1],[3,4]],[[5,2],[-2,1]])==[[12,3],[7,10]]
    # Free noncommutative polynomials: distinguish a*e from e*a.
    sym=lambda s:{s:1}
    def plus(*terms):
        out=Counter()
        for sign,v in terms:
            for key,c in v.items():out[key]+=sign*c
        return {k:v for k,v in out.items() if v}
    def mul(a,b):
        out=Counter()
        for x,c in a.items():
            for y,d in b.items():out[x+y]+=c*d
        return dict(out)
    a,b,c,d,e,f,g,h=map(sym,'abcdefgh')
    M1=mul(plus((1,a),(1,d)),plus((1,e),(1,h)))
    M2=mul(plus((1,c),(1,d)),e);M3=mul(a,plus((1,f),(-1,h)))
    M4=mul(d,plus((1,g),(-1,e)));M5=mul(plus((1,a),(1,b)),h)
    M6=mul(plus((1,c),(-1,a)),plus((1,e),(1,f)))
    M7=mul(plus((1,b),(-1,d)),plus((1,g),(1,h)))
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

04 分治法 · 獨立詳解 （續）

```
    assert plus((1,M1),(1,M4),(-1,M5),(1,M7))=={'ae':1,'bg':1}
    assert plus((1,M3),(1,M5))=={'af':1,'bh':1}
    assert plus((1,M2),(1,M4))=={'ce':1,'dg':1}
    assert plus((1,M1),(-1,M2),(1,M3),(1,M6))=={'cf':1,'dh':1}
    for L in range(12):
        n=2**L
        if L==0:t=1
        else:t=5*t+n*n
        assert t==5*5**L-4*n*n
    rounding=0
    from fractions import Fraction
    for b in [Fraction(6,5),Fraction(3,2),Fraction(2),Fraction(7,3)]:
        C=b/(b-1)
        for n in range(1,501):
            for use_ceil in [False,True]:
                q=n
                for j in range(20):
                    assert abs(q-Fraction(n,b**j))<=C
                    x=Fraction(q,b);q=ceil(x) if use_ceil else
x.numerator//x.denominator;rounding+=1
    return
{'array_cases':arrays,'recursive_matrix_pairs':matrices,'noncommutative_output_identities':4,
'rounding_states':rounding,'sample_crossing_one_based':[4,7,6],'strassen_sample':[[12,3],
[7,10]],'scope':'Finite examples and symbolic identities only; induction and complexity
proofs reside in the chapter.'}

if __name__=='__main__':
    import json
    print(json.dumps(check(),ensure_ascii=False,indent=2))
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

04 分治法 • 獨立詳解 （續）

原創驗算程式附錄: maximum_subarray.py

```
"""Compare divide-and-conquer and linear maximum-subarray algorithms."""

def crossing(a, low, mid, high):
    left_sum = float("-inf")
    total = 0
    for i in range(mid, low - 1, -1):
        total += a[i]
        left_sum = max(left_sum, total)
    right_sum = float("-inf")
    total = 0
    for j in range(mid + 1, high + 1):
        total += a[j]
        right_sum = max(right_sum, total)
    return left_sum + right_sum


def divide(a, low=0, high=None):
    if high is None:
        high = len(a) - 1
    if low == high:
        return a[low]
    mid = (low + high) // 2
    return max(divide(a, low, mid), divide(a, mid + 1, high), crossing(a, low, mid, high))


def linear(a):
    best_end = best = a[0]
    for value in a[1:]:
        best_end = max(value, best_end + value)
        best = max(best, best_end)
    return best


sample = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
assert divide(sample) == linear(sample) == 6
print(divide(sample))
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study